

FinResearch AI

Multi-Agent Financial Research System

Evaluation Presentation: architecture, agent logic, code walkthrough, live demo

Prepared for project evaluation - beginner friendly, from scratch to full detail

October 2026

Demo data is illustrative sample data prepared for product demonstration only.

- > 1. The problem and the solution in one sentence
- > 2. Complete project overview (the end-to-end journey)
- > 3. Technology map: which technology for which feature - and why NOT the alternative
- > 4. The seven agents: logic, code to show, and where the code lives
- > 5. Data model and API surface (what holds it together)
- > 6. Live demo script: exactly what to click on the live website
- > 7. Code walkthrough: which files to open and what to say
- > 8. Security, reliability, cost and testing
- > 9. Limitations (honest) and future scope
- > 10. Evaluator Q&A cheat-sheet

- **Analysing one company properly takes hours: read 100+ pages, copy numbers, scan fine print for warning signs, compare peers, write it up.**

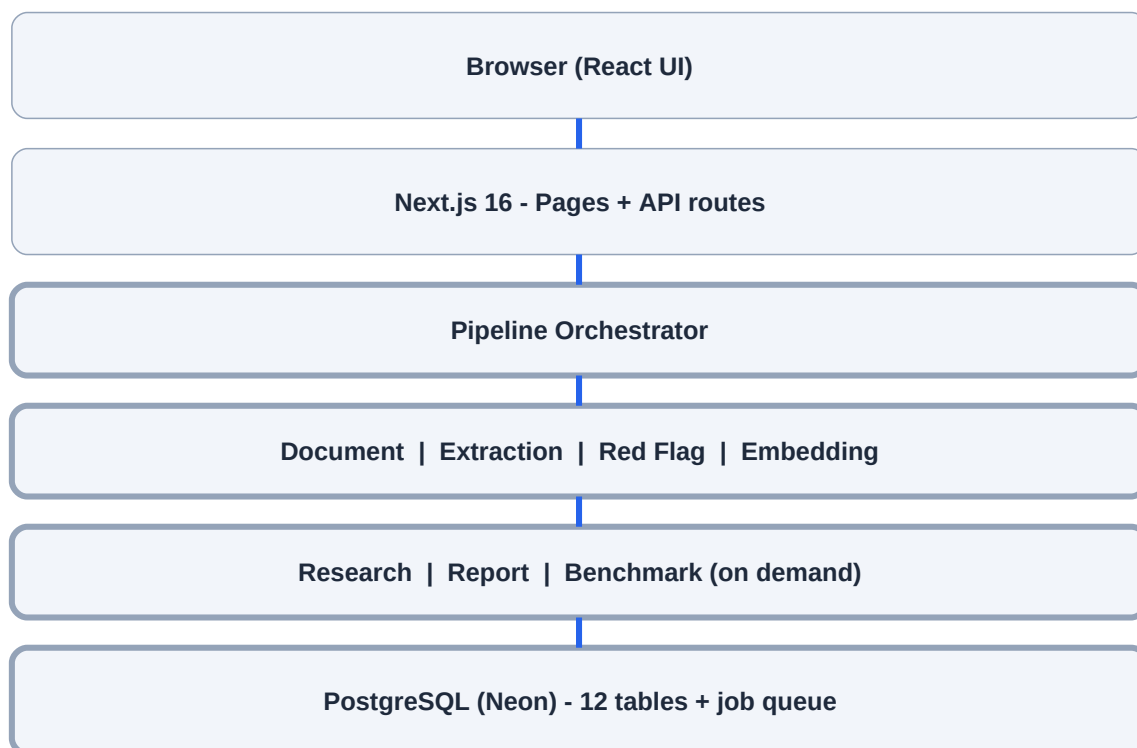
- > Most AI chat tools are fast but UNTRUSTWORTHY for finance: they invent numbers, show no sources, and forget your documents between chats.

- > Analysts, students and investors need both: the speed of AI AND the evidence trail of a spreadsheet.

- > Also: the tooling is fragmented - extraction in one app, comparison in another, report writing in a third.

- **Our one-sentence goal: upload a filing -> get trustworthy, cited financial analysis in minutes.**

- > Upload financial documents (PDF/DOCX/TXT) per company and fiscal year.
- > AUTOMATIC: text indexing (Document Agent) -> 26 metrics with evidence (Extraction Agent) -> risk flags with quotes (Red Flag Agent) -> optional semantic index (Embedding Agent).
- > ON DEMAND: ask research questions with [S1]-style citations (Research Agent), compare companies (Benchmark Agent), generate an analysis report (Report Agent).
- > Every number and every claim carries its source text. If AI is unavailable, deterministic fallbacks keep every feature alive.
- > Runs entirely on free infrastructure: Render + Neon + Groq.



> Upload is fast because processing is a BACKGROUND JOB (durable queue in Postgres) - no request timeouts.

External services

- Groq AI (LLM): extract, flag, research, report, benchmark narratives
- Embeddings provider (optional): Gemini / Jina / Ollama via 3 env vars
- Render Cron (15 min): drains the job queue
- Secrets stay server-side (env vars only)

Technology	Feature it powers	Why we chose it	Why NOT the alternative
Next.js 16 App Router	Whole web app (pages + API)	One codebase, one deploy, SSR, file routing	Vite React: needs a separate backend (2x deploy, CORS/auth pain)
React 19 + TypeScript	All interactive UI	Components + type safety at this complexity	Vanilla JS: unmanageable; plain JS: errors reach production
Tailwind CSS 4	All styling	Fast, consistent, tiny CSS payload	Bootstrap/MUI: heavy bundles, hard to customise
Recharts	Dashboard + benchmark charts	Declarative React charts from state	Chart.js: imperative canvas, awkward with React
Framer Motion	Animations	Declarative motion for UI polish	Raw CSS transitions: clumsy for sequences
react-dropzone + react-hot-toast	Upload UX + notifications	Accessible DnD; tiny toast library	Hand-rolled DnD has browser quirks; full alert suites are overkill

Technology	Feature it powers	Why we chose it	Why NOT the alternative
Next.js API Routes	20 REST endpoints	Same deploy unit as UI; no CORS; server-side secrets	Express/Django: second service to deploy, auth must be shared
PostgreSQL (Neon)	12-table data model	Relational analytics for finance; ACID; free serverless tier	MongoDB: weak joins/aggregation for company-period analysis
Drizzle ORM	All queries + migrations	TypeScript-first, SQL-like, light, safe (parameterised)	Prisma: heavier runtime/codegen; raw SQL: injection risk
JSONB vectors	Semantic search storage	Vectors in plain Postgres, cosine computed in-app; zero setup	pgvector: better at big scale, needs extension + tuning (roadmap)
NextAuth + bcryptjs	Accounts and sessions	Battle-tested auth; salted slow hashes; HTTP-only cookies	Clerk/Auth0: paid at scale; hand-rolled JWT: security risk
unpdf + pdf-parse	PDF text extraction	Two robust parsers (one as fallback), pure JS	Puppeteer/OCR: hundreds of MB RAM - impossible on free tiers

Decision	Why we do it THIS way	Why NOT the alternative
Hybrid deterministic + AI	Numbers must be exact: regex/thresholds extract metrics and flags; AI writes language and adds findings	Pure-AI invents numbers; pure-rules cannot narrate
Custom agents (not LangChain)	Each agent = one readable file with explicit I/O, own fallback, own tests - explainable in a viva	LangChain hides flow behind abstractions; heavy dependencies; hard to debug
Groq (openai/ gpt-oss-120b)	Free tier, very fast inference, OpenAI-compatible API	OpenAI: paid. Gemini: slower + confusing key errors. Local LLM: needs GPU
Provider-agnostic embeddings	3 env vars point at ANY OpenAI-compatible endpoint (Gemini/Jina/Ollama)	Hard-coded vendor = lock-in + single point of failure
Grounded answers	Research validates that numbers in answers appear in source excerpts; otherwise return verbatim evidence	Blindly trusting an LLM destroys credibility
Deterministic report tables	Key Financials rendered from stored metrics by code	LLM re-typing numbers = transcription errors

Technology	Purpose	Why / why not the alternative
Render free web service	Hosts the Next.js app; auto-deploys from GitHub	Free + simple for students. Vercel hobby had function/body quirks for this workload; a VPS costs money
Neon serverless Postgres	Database (pooled URL for app, direct URL for DDL)	Free, autosuspend, pooling built in. Local/RDS Postgres: not free / not reachable
Render Cron (15 min)	Calls the protected worker endpoint to drain job retries	Queue services (SQS, Redis/BullMQ): extra accounts, cost and failure points
Node 22 via .nvmrc	Pinned runtime	Reproducible builds. Floating versions cause 'works on my machine' bugs
GitHub + PRs	Version control and deploy trigger	Reviewable history; PR workflow protects main. Zip-over-email is not version control
Total infrastructure cost	Rs 0 / \$0 per month on free tiers	The right kind of project for a student budget

Agent	One-line job	Runs when	Code file
1. Document Agent	Parse, section, chunk and index the text	On upload (pipeline)	src/lib/agents/ documentAgent.ts
2. Extraction Agent	Extract 26 metrics + evidence lines	On upload (pipeline)	src/lib/agents/ extractionAgent.ts + extractionUtils.ts
3. Red Flag Agent	Find risks: regex, thresholds, trends, AI	On upload (pipeline)	src/lib/agents/ riskAgent.ts + analysisUtils.ts
4. Embedding Agent	Optional semantic vectors for search	On upload (pipeline, optional)	src/lib/agents/ embeddingAgent.ts
5. Research Agent	Cited answers from YOUR documents	When the user asks	src/lib/agents/ researchAgent.ts
6. Report Agent	Structured analysis report + disclaimer	When the user asks	src/lib/agents/ reportAgent.ts
7. Benchmark Agent	Company-vs-company comparisons	When the user asks	src/lib/agents/ benchmarkAgent.ts
Orchestrator	Runs pipeline stages + job queue	Always	src/lib/agents/ orchestrator.ts

- Presenter line: 'Each agent is one file, one job, one fallback - that is why the system degrades gracefully instead of crashing.'

- > Logic: text -> section detection (7 heading patterns) -> sliding-window chunks (1500 chars, 200 overlap) -> document_chunks rows with section + position.
- > Why overlap: a sentence on a chunk boundary must survive whole in at least one chunk - quotes stay complete.
- > Hybrid search built in: keyword relevance always works; cosine similarity over embeddings when available.

FOLDER/FILE: `src/lib/agents/documentAgent.ts`

Show live: Document detail page -> chunks panel

```
export function chunkText(text: string, chunkSize = 1500, overlap = 200): string[] {
  const chunks: string[] = [];
  let start = 0;
  while (start < text.length) {
    chunks.push(text.slice(start, Math.min(start + chunkSize, text.length)));
    start += chunkSize - overlap;
  }
  return chunks;
}
```

What to say: 'This is a sliding window - step size is chunkSize minus overlap, so context never gets surgically cut.'

- > Hybrid logic (this is the key talking point): (1) deterministic LABELS regex parses 'Total revenue: \$383,285 million' style lines and keeps the matched line as evidence; (2) an LLM pass reads the head+tail of the document (15k chars each) for context; (3) mergeWithLocalMetrics lets the exact regex values win.
- > Normalisation: percents to decimals, money parsed with units, fiscal year detected from labelled lines.
- > If AI is down: step (1) alone still produces the metrics table - the page never goes empty.

FOLDER/FILE: `src/lib/agents/extractionAgent.ts + extractionUtils.ts`

Show live: Document detail -> Metrics (26 rows with 'evidence' source line)

```
// extractionUtils.ts -- labelled grammar (excerpt)
revenue: /\s*(?:total net sales|total sales|total net revenues?|
    total revenues?|net sales|net revenues?|revenues?)\s*[:\u2014-]/i

// extractionAgent.ts -- hybrid merge (excerpt)
extracted = mergeWithLocalMetrics(verified, extractMetricsLocally(content));
```

What to say: 'Regex is used where precision matters; AI is used where language matters. The evidence column proves each number to the audience.'

- > Layer 1 - regex red flags with severity: auditor qualification (critical), going concern (critical), accounting control (high), liquidity/covenant/default (high). Each stores the matched SOURCE TEXT.
- > Layer 2 - metric anomalies: negative margins, collapsing growth, negative free cash flow (threshold maths on extracted metrics).
- > Layer 3 - cross-period trends: compare with the previous fiscal year of the SAME company (declining margins, rising debt...) and cite BOTH documents.
- > Layer 4 - AI review adds material risks the rules missed - instructed not to duplicate deterministic findings.

FOLDER/FILE: src/lib/agents/riskAgent.ts + analysisUtils.ts

Show live: Risks page: Tesla FY2026 -> four flags with exact filing quotes

```
// analysisUtils.ts -- deterministic detector (excerpt)
{
  pattern: /\b(?:substantial doubt about.{0,100}going[- ]concern|going[- ]concern)\b/i,
  riskType: "Going Concern", severity: "critical",
  title: "Going-concern disclosure present",
}
```

What to say: 'Four regex families cover the worst disclosures in audit language. Regex first = zero hallucination risk for the most dangerous claims.'

- > Batches 32 chunks at a time to ANY OpenAI-compatible /embeddings endpoint (3 env vars: EMBEDDING_BASE_URL / EMBEDDING_API_KEY / EMBEDDING_MODEL).
- > Vectors stored in JSONB with the model name; cosine similarity computed in app memory (no pgvector needed).
- > Model versioning: change the model -> chunks re-embed automatically on the next run.
- > No provider -> 'unavailable' with a clear log; research falls back to keyword search. Nothing else breaks.
- > Real-world lesson from this project: Ollama works on your laptop (localhost), a cloud deployment needs a cloud endpoint (Gemini/Jina) - same code, different env vars.

FOLDER/FILE: `src/lib/agents/embeddingAgent.ts + llmClient.ts`

Show live: Document detail -> Build embeddings -> status becomes 'completed'

```
// llmClient.ts -- provider-agnostic (excerpt)
await fetch(`${baseUrl}/embeddings`, {
  method: "POST",
  headers: { "content-type": "application/json", authorization: `Bearer ${apiKey}` },
  body: JSON.stringify({ model, input: inputs }),
});
```

What to say: 'One request format, any provider - that is dependency inversion at the HTTP level.'

- > Flow: decompose compound question into retrieval steps -> search per step (semantic if embeddings exist, keyword otherwise) -> relevance filter -> buildCitations [S1], [S2]... (document + section + chunk + excerpt).
- > The LLM synthesises ONLY from retrieved passages; validateGroundedResearchAnswer then checks that numbers in the answer appear in the source excerpts.
- > Any AI failure -> return the verbatim passages labelled as EVIDENCE (honest fallback), never a guess.
- > The 'reasoning' shown in the UI lists the retrieval steps that ran - transparency, not hidden chain-of-thought.

FOLDER/FILE: `src/lib/agents/researchAgent.ts + analysisUtils.ts`

Show live: Research page: the demo prompts (value / evidence / comparison / follow-up)

```
// researchAgent.ts -- citation ids (excerpt)
citationId: `${index + 1}`

// what the user sees in the answer:
//   ...operating margin was 44.1% [S1]...
//   [S1] Apple FY2023 -- Results of Operations, chunk 2: "..."
```

What to say: 'Every claim is checkable - the citation opens the exact chunk it came from.'

- > Sections: Key Financials, Business Overview, Risk Assessment, Financial Analysis, Outlook, Disclaimer.
- > Key Financials table is rendered BY CODE from stored metrics (renderKeyFinancialsSection) - the LLM never re-types numbers.
- > Narrative sections come from the LLM; on failure createLocalReport produces a complete deterministic report.
- > describeNarrativeFallbackReason converts raw AI errors into clean user guidance (this project's bug-fix story: stale GEMINI_API_KEY advice used to leak into reports).
- > Reports are downloadable from the Reports page (text/markdown export).

FOLDER/FILE: `src/lib/agents/reportAgent.ts + aiStatus.ts`

Show live: Reports page -> generate -> show table + disclaimer -> Download

```
// reportAgent.ts -- table stays deterministic (excerpt)
function renderKeyFinancialsSection(profiles) { /* builds the table from stored metrics */ }
// aiStatus.ts -- clean disclaimer mapping (excerpt)
export function describeNarrativeFallbackReason(error?: unknown) { /* maps to Groq guidance */ }
```

What to say: 'Numbers by code, prose by AI - division of labour that eliminates the classic AI typo risk.'

- > Builds a metric profile per company (latest or chosen fiscal period), normalises units, then compares: revenue/growth, profitability (margins, ROE), leverage/liquidity (D/E, current ratio), risk counts.
- > Deterministic core: ranking (e.g. by ROE) and N/A handling work without AI.
- > AI narrative is instructed to name the fiscal period beside every figure, refuse silent year-mismatch comparisons, and never give buy/sell advice.

FOLDER/FILE: `src/lib/agents/benchmarkAgent.ts`

Show live: Benchmark page: Apple + Microsoft + Tesla side by side

```
// benchmarkAgent.ts -- deterministic ranking (excerpt)
const ranked = [...context].sort((a, b) => Number(b.metrics?.roe || 0) - Number(a.metrics?.roe || 0));
```

What to say: 'Even with the AI switched off, this page still produces a correct ranking.'

- > Failure isolation: extraction errors do NOT block risk scanning; embeddings NEVER block anything; the document lands 'completed' or 'partial' with the failing stage named.
- > Durable queue in Postgres: upload inserts a document_processing_jobs row; the worker claims it ATOMICALLY.
- > Crash safety: 15-minute stale-lock expiry lets another worker retry; retries use next_attempt_at backoff up to max attempts; batch size 5 per run.
- > The worker endpoint is protected by CRON_SECRET and runs on a 15-minute Render cron (plus an immediate kick after upload for demo speed).

```
// orchestrator.ts -- atomic claim (excerpt)
await db.update(documentProcessingJobs)
  .set({ status: 'processing', attempts: sql`attempts + 1`, lockedAt: now })
  .where(and(
    eq(documentProcessingJobs.id, jobId),
    or(
      and(eq(status, 'queued'), lte(nextAttemptAt, now), lt(attempts, maxAttempts)),
      and(eq(status, 'processing'), lt(lockedAt, staleBefore)) // crashed run recovery
    ),
  ));
```

Group	Tables	What they hold
Identity	users, research_sessions	Accounts (bcrypt hashes) and scoped research workspaces
Content	companies, documents, document_chunks	Company master data, uploaded files, searchable chunks (+ JSONB vectors)
Analysis	financial_metrics, risk_flags, analysis_reports, chat_messages, benchmark_comparisons	26 metrics + evidence, risks + source quotes, generated reports, cited Q&A, saved benchmarks
Operations	document_processing_jobs, agent_logs	The durable job queue and the per-agent audit trail shown in the UI

> Design points evaluators like: evidence and citations are stored AS DATA (JSONB), not just displayed; every analysis row is tied to a document and a user; agent_logs makes the whole pipeline auditable after the fact.

Step	Where (live website)	What happens / what to say
1	Register / Login	Clean auth flow. 'Passwords are bcrypt-hashed; sessions are HTTP-only cookies.'
2	Documents -> Upload Apple_Annual_Report_FY2023.pdf (company Apple, ticker AAPL, FY 2023)	Status moves through processing -> completed. 'The upload is fast because agents run as a background job.'
3	Open the document -> agent activity trail	Four stages logged with durations. 'This is the orchestrator's audit trail - every stage is observable.'
4	Document -> Metrics panel	26 metrics with evidence lines. 'Numbers came from deterministic parsing; here is the source line for each.'
5	Upload Tesla_Annual_Report_FY2026.pdf -> Risks page	Four flags (auditor qualification, going concern, accounting control, liquidity) with exact quotes. 'Regex detectors - zero hallucination risk on the most dangerous claims.'

Step	Where (live website)	What happens / what to say
6	Upload Tesla FY2024 then FY2025 -> Risks page	Cross-period trend findings appear citing BOTH documents. 'Trends are computed by code across periods.'
7	Document detail -> Build embeddings	Semantic index builds (needs a configured provider). 'Same button works with Ollama locally and Gemini in the cloud.'
8	Research page	Ask: 'What evidence supports the going concern risk?' -> answer with [S1] quotes. 'Every sentence is checkable.'
9	Benchmark: Apple + Microsoft + Tesla	Side-by-side table + charts; Tesla shows the risk spike. 'Periods are named; mismatched years are refused.'
10	Reports -> generate for Tesla	Key Financials table + risk narrative + disclaimer. 'Table is rendered by code from stored metrics - the AI never re-types numbers.'
11	/api/health	JSON shows AI configured, models, embedding status. 'Live configuration transparency.'

Code walkthrough (1/2) - open these files in this order

What to show and what to say about each

File to open	Function to point at	What to say (30 seconds each)
src/lib/agents/ orchestrator.ts	runDocumentPipeline	Stage order + failure isolation + 'partial' status design
src/lib/agents/ documentAgent.ts	chunkText + extractSections	Sliding window with overlap; regex section detection
src/lib/agents/ extractionUtils.ts	LABELS + extractMetricsLocally	The labelled grammar; evidence capture; numbers normalisation
src/lib/agents/ analysisUtils.ts	detectTextualRedFlags	The four deterministic patterns and severities
src/lib/agents/ extractionAgent.ts	extractFinancialMetrics	Hybrid merge: regex values win over AI re-typing

File to open	Function to point at	What to say (30 seconds each)
src/lib/agents/ researchAgent.ts	buildCitations + buildLocalResearchAnswer	[S1] citations; verbatim-evidence fallback instead of guessing
src/lib/agents/ analysisUtils.ts	validateGroundedResearchAnswer	Numbers in answers must appear in source excerpts
src/lib/agents/ embeddingAgent.ts	indexDocumentEmbeddings	Batching, model versioning, optional-by-design
src/lib/agents/ reportAgent.ts	renderKeyFinancialsSection + createLocalReport	Deterministic tables; deterministic fallback report
src/lib/agents/aiStatus.ts	describeNarrativeFallbackReason	User-safe error mapping (the GEMINI->Groq bug-fix story)
src/app/api/documents/route.ts	PDF extraction block	Two-parser fallback + actionable error messages

- > Passwords: bcryptjs (salted, slow hashes). Auth: NextAuth with HTTP-only cookies. No third-party lock-in.
- > Data isolation: every query is scoped to `session.user.id` - users cannot read each other's documents, chats or reports.
- > Injection-safe: all SQL through Drizzle ORM (parameterised); server-side validation for UUIDs, file types and sizes.
- > Secrets (GROQ_API_KEY, CRON_SECRET, SEED_SECRET, EMBEDDING_API_KEY) exist only in server environment variables - nothing sensitive in the browser bundle.
- > Protected operations: worker and seed endpoints require Bearer tokens; cron calls are authenticated.
- > Safe output: AI errors are sanitised before display; reports carry an explicit 'not investment advice' disclaimer.
- > Demo integrity: all sample documents are clearly labelled 'SAMPLE DOCUMENT - illustrative figures'.

If this fails...	The user sees...	Everything that still works
Embeddings provider missing/down	'unavailable' + explanation (not an error)	Keyword research, metrics, risks, reports
Groq AI down / key invalid	Deterministic report + clean disclaimer	Regex metrics + flags, keyword evidence search
One pipeline stage throws	Status 'partial' naming the stage	Later stages still run; activity log explains
Corrupt or scanned PDF	422 naming the exact parse reason	All other documents and features
Cloud Ollama misconfig (localhost)	Embedding 'failed' with connection error	Everything except semantic search

- > Upload speed: processing is asynchronous (job queue) - the browser never waits for AI calls.
- > AI budget control: prompts take head+tail of documents (15k chars each) instead of whole files; embedding batches of 32; 8k chars max per chunk.
- > Storage: JSONB vectors + in-app cosine suit small/medium corpora; pgvector is the scale-up path (roadmap).
- > Free-tier aware: Render free cold starts (~1 min) are mitigated with an uptime monitor; Neon autosuspends when idle.
- > Total monthly infrastructure cost for the demo: Rs 0 / \$0 (Render free + Neon free + Groq free).

Suite	What it proves to an evaluator
extractionUtils.test.ts	The metric grammar parses real filing styles and stores evidence
analysisUtils.test.ts	Red-flag patterns, thresholds, trend detection and similarity maths are correct
aiStatus.test.ts	Fallbacks name the right provider key; retired advice (GEMINI) never leaks
llmClient.test.ts	AI/Embeddings request shaping and error mapping
validation.test.ts / dbConfig.test.ts	Input validation and pooled-vs-direct connection logic
embeddingUiState.test.ts / seedPlan.test.ts	Embedding status UI logic; deterministic demo seeding

Limitation today	Why it is acceptable	Future scope
No OCR for scanned/image PDFs	Clear, actionable error instead of silent garbage	OCR pipeline (Tesseract) for scanned filings
Vectors in JSONB + in-app cosine	Correct and simple at small/medium scale	pgvector + HNSW index for large corpora
Embeddings need an external provider	Keyword mode keeps ALL features usable	Ship a default provider / local model option
No live market prices	Analysis is document-based and traceable	Optional quote/valuation API integration
AI can still err (rarely)	Grounding validation + citations + disclaimers	Stronger claim-level verification; streaming UX
Single-user workspaces	Fits the student/analyst use case	Organisations, roles, sharing; Docker + CI/CD

Question they ask	The answer to give
Why multi-agent instead of one big prompt?	Each agent has one job, one test surface and one fallback. Failures stay isolated, the pipeline is auditable (agent_logs), and precision work stays deterministic.
How do you control hallucination?	Three defences: deterministic extraction for numbers; citations with source excerpts; grounding validation that checks numbers appear in retrieved text. Worst case: verbatim evidence instead of an answer.
Why not LangChain / CrewAI?	This scale does not need them. Explicit TS files are easier to explain, test, debug and deploy; fewer dependencies, smaller bundle.
What if the AI provider dies?	Every feature has a deterministic fallback (regex metrics/flags, keyword search, local report/benchmark). The UI explains the mode instead of failing.

Question they ask	The answer to give
Is it secure?	Bcrypt hashes, HTTP-only sessions, per-user query scoping, ORM parameterisation, server-side secrets, Bearer-protected worker/seed endpoints, sanitised error messages.
How does it scale?	Async jobs + batching + prompt trimming cover the free tier; the documented scale-up path is pgvector, queue hardening and caching. Design decisions are deliberate, not accidental.
What did YOU learn building it?	Mention: the GEMINI->Groq disclaimer bug (trust!), the 'saved 404 page as PDF' debugging story (errors must self-diagnose), and localhost-vs-cloud Ollama (environment config matters).
Is this financial advice?	No - it is research assistance with evidence. Every report carries a disclaimer; benchmark refuses buy/sell language by design.
What would you add with 2 more weeks?	OCR, pgvector, streaming chat, Docker+CI, multi-user roles, and a chart-heavy valuation module.

Thank you

FinResearch AI - trustworthy financial research, one upload away.

Questions welcome. Live demo available at the deployed website.

Documentation: [PROJECT_DOCUMENTATION.pdf](#) | Demo data: [demo-test-data/](#) (28 sample annual reports)